Client / Freelancer
:ForgotPasswordView
:AuthRoute
:AuthController
:AuthService
:AuthRepo
:PrismaService
Database
1. Enter email, click "Send code"
2. POST /api/auth/otp
3. Send request by post method
4. sendOTP(dto)
5. findUserByEmail(email)
6. user.findUnique({ where: { email, deletedAt: null } })
7. Execute query
8. Return result
9. Return result
10. Return result
11. Handle account verification result
alt
[Account is null]
11.1. Return error 422 EMAIL_NOT_FOUND
11.1.1. Return error
11.1.2. Return error
11.1.3. Display error message
[Account exists]
12. findVerificationCode ByEmailAndType(email,type)
13. verificationCode.findUnique({ email_type })
14. Execute query
15. Return result
16. Return result
17. Return result
18. Handle check OTP attempts
alt
[Too many attempts (attempts > 5)]
18.1. Return error 422 TOO_MANY_ATTEMPTS
18.1.1. Return error
18.1.2. Return error
18.1.3. Display error message
[Attempts <= 5]
19. generateOTP(), set expiresAt, attempts++
20. upsertVerificationCode(data)
21. verificationCode.upsert(...)
22. Execute upsert
23. Return result
24. Return result
25. Return result
26. Handle save OTP result
alt
[Save OTP not success]
26.1. Return error
26.1.1. Return error
26.1.2. Return error
26.1.3. Display error message
[Save OTP success]
26.2. Send OTP email via Gmail SMTP
26.2.1. Return success
26.2.2. Return success
26.2.3. Return success
26.2.4. Display OTP sent message
27. Enter code and new password, click "Reset password"
28. Validate input fields
opt
[Invalid data]
28.1. Display error message
29. POST /api/auth/forgot-password
30. Send request by post method
31. forgotPassword(payload)
32. findUserForLogin(email)
33. user.findUnique({ where: { email }, include: { userRoles } })
34. Execute query
35. Return result
36. Return result
37. Return result
38. Handle user validation result
alt
[Account is null]
38.1. Return error 422 EMAIL_NOT_FOUND
38.1.1. Return error
38.1.2. Return error
38.1.3. Display error message
[User is banned]
38.2. Return error 403 USER_BANNED
38.2.1. Return error
38.2.2. Return error
38.2.3. Display error message
[Account is valid]
39. validateVerificationCode({ email, code,type: PASSWORD_RESET })
40. verificationCode.findFirst({ code, email, type })
41. Execute query
42. Return result
43. Return result
44. Return result
45. Handle OTP verification result
alt
[Verification code not found]
45.1. Return error 422
45.1.1. Return error
45.1.2. Return error
45.1.3. Display error message
[Verification code expired]
45.2. Return error 422
45.2.1. Return error
45.2.2. Return error
45.2.3. Display error message
[Verification code valid]
46. hash(newPassword)
par
47. update(password)
48. user.update({ where, data: { password } })
49. Execute update
50. Return result
51. Return result
52. Return result
53. deleteVerifycation Code(PASSWORD_RESET)
54. verificationCode.delete(...)
55. Execute delete
56. Return result
57. Return result
58. Return result
59. Return success message
60. Return success message
61. Return success message
62. Display "Update password successfully"